МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №7

з дисципліни **“Функціональне програмування”**

на тему **“FUNCTOOLS TA OPERATOR”**

Варіант 13. Мобільні тарифні плани

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: опанувати часткове застосування (`functools.partial`), згортку (`functools.reduce`), поліморфізм через `functools.singledispatch`, повний порядок через `functools.total_ordering` та гетери модуля `operator`; побудувати конвеєр відбору мобільних тарифів і поліморфний API завантаження даних.

Теоретичні відомості:

`partial` фіксує частину аргументів функції й повертає нову функцію з вужчим інтерфейсом — так із загальної `score(row, w_minutes=…, w_data=…)` виходить готовий «оцінювач» для конвеєра. `reduce` послідовно застосовує акумулятор до колекції; з ним універсальний `pipeline(value, *funcs)` — один рядок: кожна наступна функція отримує результат попередньої. Модуль `operator` дає оптимізовані заміни lambda-гетерів: `itemgetter` для словників, `attrgetter` для об'єктів (із кількома полями повертає кортеж, зручний для багатокритеріального сортування), `methodcaller` для викликів методів.

`singledispatch` перетворює звичайну функцію на генеричну: реалізації реєструються під тип першого аргументу, тож одна точка входу однаково приймає JSON-рядок, шлях, відкритий файл чи готовий список. `total_ordering` добудовує всі оператори порівняння з `__eq__` та одного строгого (`__lt__`); для чужих типів методи повертають `NotImplemented`, щоб Python коректно відмовив у порівнянні.

Хід роботи:

1. У `core.py` реалізував універсальний `pipeline` через `reduce` та обгортки `mapf` / `filt` як `partial(map, …)` і `partial(filter, …)`, щоб кроки конвеєра були готовими функціями над ітерованим:

```python
T = TypeVar("T")

Row = Mapping[str, Any]


def pipeline(value: T, *funcs: Callable[[Any], Any]) -> Any:
    """Протягує value через funcs зліва направо: pipeline(x, f, g) == g(f(x))."""
    return reduce(lambda acc, f: f(acc), funcs, value)


def mapf(fn: Callable[[Any], Any]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """partial-обгортка над map: mapf(fn)(iterable)."""
    return partial(map, fn)


def filt(pred: Callable[[Any], bool]) -> Callable[[Iterable[Any]], Iterable[Any]]:
    """partial-обгортка над filter: filt(pred)(iterable)."""
    return partial(filter, pred)
```

2. За варіантом 13 описав трансформації тарифів: `normalize_tariff` очищує назву через `itemgetter` і `methodcaller("strip")` / `methodcaller("title")` та

приводить типи; `meets_requirements` і `score` мають keyword-параметри, які далі фіксуються через `partial`; `with_score` та `to_tariff` повертають нові значення:

```python
get_name = itemgetter("name")
strip, title = methodcaller("strip"), methodcaller("title")


def normalize_tariff(row: Row) -> dict[str, Any]:
    """Новий запис із очищеною назвою та числовими полями."""
    return {
        "name": title(strip(str(get_name(row)))),
        "minutes": int(row["minutes"]),
        "data_gb": float(row["data_gb"]),
        "sms": int(row["sms"]),
        "price": float(row["price"]),
    }


def meets_requirements(
    row: Row, *, min_minutes: int = 0, min_data: float = 0.0, min_sms: int = 0
) -> bool:
    """Предикат: тариф задовольняє вимоги користувача (кожну окремо)."""
    return bool(
        row["minutes"] >= min_minutes
        and row["data_gb"] >= min_data
        and row["sms"] >= min_sms
    )


def score(
    row: Row, *, w_minutes: float = 0.1, w_data: float = 10.0, w_sms: float = 0.05
) -> float:
    """Цінність тарифу: зважена корисність на одиницю ціни."""
    value = row["minutes"] * w_minutes + row["data_gb"] * w_data + row["sms"] * w_sms
    return value / row["price"] if row["price"] > 0 else 0.0


def with_score(row: Row, scorer: Callable[[Row], float]) -> dict[str, Any]:
    """Новий запис із полем score, обчисленим scorer."""
    return {**row, "score": scorer(row)}


def to_tariff(row: Row) -> Tariff:
    """Перетворює словник на незмінний Tariff."""
    return Tariff(
        name=row["name"],
        minutes=row["minutes"],
        data_gb=row["data_gb"],
        sms=row["sms"],
        price=row["price"],
        score=row["score"],
    )
```

3. Функція `rank_tariffs` збирає конвеєр: фільтр і оцінювач створюються через `partial`, сортування — два стабільних проходи з `attrgetter("price", "name")`

та `attrgetter("score")` (score спадно, ціна зростом, назва зростом), потім Top-N. `total_price` — reduce-агрегація:

```python
def rank_tariffs(
    rows: Iterable[Row],
    *,
    top: int = 3,
    min_minutes: int = 0,
    min_data: float = 0.0,
    min_sms: int = 0,
    w_minutes: float = 0.1,
    w_data: float = 10.0,
    w_sms: float = 0.05,
) -> list[Tariff]:
    """Конвеєр: нормалізація → фільтр за вимогами → score → сортування → Top-N."""
    accept = partial(
        meets_requirements, min_minutes=min_minutes, min_data=min_data, min_sms=min_sms
    )
    scorer = partial(score, w_minutes=w_minutes, w_data=w_data, w_sms=w_sms)
    result: list[Tariff] = pipeline(
        rows,
        mapf(normalize_tariff),
        filt(accept),
        mapf(partial(with_score, scorer=scorer)),
        mapf(to_tariff),
        partial(sorted, key=attrgetter("price", "name")),
        partial(sorted, key=attrgetter("score"), reverse=True),
        lambda it: list(it)[:top],
    )
    return result


def total_price(tariffs: Iterable[Tariff]) -> float:
    """reduce-агрегація: сумарна ціна вибраних тарифів."""
    return reduce(lambda acc, t: acc + t.price, tariffs, 0.0)
```

4. Базовий приклад з методички (студенти) реалізував тим самим інструментарієм: `clean_and_rank` нормалізує записи, відбирає повнолітніх і сортує за score ↓, name ↑:

```python
def normalize_student(row: Row) -> dict[str, Any]:
    """Новий запис студента: name у Title Case, age → int, score → float."""
    return {
        "name": title(strip(str(get_name(row)))),
        "age": int(row["age"]),
        "score": float(row["score"]),
    }


def clean_and_rank(rows: Iterable[Row], top: int = 3) -> list[dict[str, Any]]:
    """Нормалізувати → повнолітні → score ↓, name ↑ → Top-N."""
    result: list[dict[str, Any]] = pipeline(
        rows,
        mapf(normalize_student),
        filt(lambda r: r["age"] >= 18),
        partial(sorted, key=itemgetter("name")),
        partial(sorted, key=itemgetter("score"), reverse=True),
        lambda it: list(it)[:top],
    )
    return result
```

5. У `models.py` описав незмінний клас `Tariff` з `@total_ordering`: ключ порівняння `(score, -price, name)`, власні `__eq__` / `__lt__` повертають

`NotImplemented` для чужих типів, `__hash__` узгоджений з рівністю:



```python
@total_ordering
@dataclass(frozen=True, eq=False)
class Tariff:
    """Мобільний тариф; порядок: score ↑, дешевший кращий, потім name."""

    name: str
    minutes: int
    data_gb: float
    sms: int
    price: float
    score: float = 0.0

    def _key(self) -> tuple[float, float, str]:
        return (self.score, -self.price, self.name)

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Tariff):
            return NotImplemented
        return self._key() == other._key()

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Tariff):
            return NotImplemented
        return self._key() < other._key()

    def __hash__(self) -> int:
        return hash(self._key())
```

6. У `loaders.py` зробив поліморфний `load_tariffs` на `singledispatch`: реалізації для `str` (JSON-рядок або шлях), `pathlib.Path` (JSON чи CSV за розширенням),

відкритого `io.TextIOBase` (формат за вмістом) і готового `list` з валідацією; базовий виклик кидає `TypeError` для непідтримуваних типів:

```python
def _parse_text(text: str) -> Rows:
    """JSON, якщо текст починається з '[', інакше CSV."""
    stripped = text.strip()
    if stripped.startswith("["):
        return _check(json.loads(stripped))
    return list(csv.DictReader(io.StringIO(stripped)))


def _check(rows: Any) -> Rows:
    """Мінімальна валідація готової колекції."""
    if not isinstance(rows, list) or any(not isinstance(r, Mapping) for r in rows):
        raise ValueError("Expected list of dicts")
    return [dict(r) for r in rows]


@singledispatch
def load_tariffs(src: object) -> Rows:
    """Повертає список словників-тарифів із JSON/CSV-джерела будь-якого типу."""
    raise TypeError(f"Unsupported type: {type(src)!r}")


@load_tariffs.register
def _(src: str) -> Rows:
    """JSON-рядок або шлях до .json/.csv."""
    if src.lstrip().startswith(("[", "{")):
        return _check(json.loads(src))
    return load_tariffs(Path(src))


@load_tariffs.register
def _(src: Path) -> Rows:
    """Файл .json або .csv за розширенням."""
    text = src.read_text(encoding="utf-8")
    if src.suffix.lower() == ".csv":
        return list(csv.DictReader(io.StringIO(text)))
    return _check(json.loads(text))


@load_tariffs.register
def _(src: io.TextIOBase) -> Rows:
    """Відкритий текстовий файл: формат визначається за вмістом."""
    return _parse_text(src.read())


@load_tariffs.register
def _(src: list) -> Rows:  # type: ignore[type-arg]
    """Готова колекція словників."""
    return _check(src)


load_users = load_tariffs
"""Базовий приклад з методички: той самий loader для записів студентів."""
```

7. Усе I/O зібрав у `main.py`: читання прикладів із JSON, CSV та відкритого файлу, друк Top-3 за вимогами з аргументів командного рядка, сума цін і

демонстрація базового прикладу:

```python
def main(argv: list[str]) -> None:
    """Друкує найкращі тарифи; аргументи: [min_minutes min_data min_sms]."""
    min_minutes = int(argv[0]) if argv else 300
    min_data = float(argv[1]) if len(argv) > 1 else 10.0
    min_sms = int(argv[2]) if len(argv) > 2 else 0

    rows_json = load_tariffs(HERE / "sample_data.json")
    rows_csv = load_tariffs(str(HERE / "sample_data.csv"))
    with open(HERE / "sample_data.json", encoding="utf-8") as f:
        rows_file = load_tariffs(f)
    print(f"json={len(rows_json)} csv={len(rows_csv)} file={len(rows_file)} rows")

    top = rank_tariffs(
        rows_json + rows_csv,
        top=3,
        min_minutes=min_minutes,
        min_data=min_data,
        min_sms=min_sms,
    )
    for t in top:
        print(
            f"{t.name:<14} {t.minutes:>5} min {t.data_gb:>5.1f} GB {t.sms:>4} sms "
            f"{t.price:>7.2f} score={t.score:.3f}"
        )
    print(f"total price of top: {total_price(top):.2f}")

    students = load_users(
        '[{"name": " alice ", "age": "19", "score": "8.5"}, '
        '{"name": "bob", "age": "17", "score": "9.1"}]'
    )
    print("students:", clean_and_rank(students, top=2))


if __name__ == "__main__":
    main(sys.argv[1:])
```

8. Написав pytest-тести: `pipeline` застосовує функції зліва направо, `mapf`/`filt` є `partial`-об'єктами, `partial` над предикатом і оцінювачем, сортування з `itemgetter` за кількома ключами, `methodcaller` та `attrgetter`:

```python
def test_pipeline_applies_left_to_right() -> None:
    assert pipeline(3, lambda x: x + 1, lambda x: x * 2, lambda x: x + 1) == 9
    assert pipeline(5) == 5


def test_mapf_filt_are_partials() -> None:
    assert isinstance(mapf(str), partial) and isinstance(filt(bool), partial)
    assert list(
        pipeline([1, 2, 3, 4], filt(lambda x: x % 2 == 0), mapf(lambda x: x * 10))
    ) == [
        20,
        40,
    ]


def test_partial_requirements_and_score() -> None:
    accept = partial(meets_requirements, min_minutes=500, min_data=10.0)
    assert accept({"minutes": 500, "data_gb": 10.0, "sms": 0}) is True
    assert accept({"minutes": 499, "data_gb": 50.0, "sms": 0}) is False
    cheap = partial(score, w_minutes=0.0, w_data=1.0, w_sms=0.0)
    assert cheap({"minutes": 0, "data_gb": 20.0, "sms": 0, "price": 10.0}) == 2.0
    assert score({"minutes": 1, "data_gb": 1.0, "sms": 1, "price": 0.0}) == 0.0


def test_operator_helpers() -> None:
    rows = [{"a": 1, "b": 2}, {"a": 1, "b": 1}, {"a": 0, "b": 9}]
    assert sorted(rows, key=itemgetter("a", "b")) == [
        {"a": 0, "b": 9},
        {"a": 1, "b": 1},
        {"a": 1, "b": 2},
    ]
    assert methodcaller("title")(methodcaller("strip")("  aLiCe ")) == "Alice"
    t = Tariff("x", 1, 1.0, 1, 10.0, 0.5)
    assert attrgetter("price", "name")(t) == (10.0, "x")
```

9. Тести конвеєра: нормалізація створює новий запис, Top-N у правильному порядку, фільтрація та тай-брейкер за назвою, reduce-агрегація, відсутність мутацій вхідних даних:

```python
def test_normalize_creates_new_record(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    out = normalize_tariff(rows[0])
    assert out == {
        "name": "Smart Start",
        "minutes": 300,
        "data_gb": 8.0,
        "sms": 50,
        "price": 150.0,
    }
    assert rows == original


def test_rank_tariffs_top_n_and_order(rows: list[dict[str, Any]]) -> None:
    top = rank_tariffs(rows, top=2, min_minutes=200, min_data=5.0)
    assert [t.name for t in top] == ["Data Max", "Talk Unlimited"]
    assert top[0].score >= top[1].score
    assert len(rank_tariffs(rows, top=10)) == 5


def test_rank_tariffs_filters_and_tiebreak() -> None:
    same = [
        {"name": "b", "minutes": 100, "data_gb": 1, "sms": 0, "price": 10},
        {"name": "a", "minutes": 100, "data_gb": 1, "sms": 0, "price": 10},
        {"name": "c", "minutes": 200, "data_gb": 2, "sms": 0, "price": 20},
    ]
    top = rank_tariffs(same, top=3)
    assert [t.name for t in top] == ["A", "B", "C"]
    assert rank_tariffs(same, top=3, min_sms=1) == []


def test_total_price_reduce(rows: list[dict[str, Any]]) -> None:
    top = rank_tariffs(rows, top=2)
    assert total_price(top) == pytest.approx(sum(t.price for t in top))
    assert total_price([]) == 0.0


def test_no_mutation(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    rank_tariffs(rows, top=5, min_minutes=100)
    assert rows == original
```

10. Тести `singledispatch`: одні й ті самі дані з JSON-рядка, шляху `str`, `Path`, відкритого файлу (JSON і CSV) та списку дають однаковий результат;

непідтримуваний тип → `TypeError`, неправильна структура → `ValueError`:

```python
JSON = '[{"name": "a", "minutes": 1, "data_gb": 1, "sms": 1, "price": 1}]'
CSV = "name,minutes,data_gb,sms,price\na,1,1,1,1\n"


def test_load_from_all_sources(tmp_path: Path) -> None:
    pj, pc = tmp_path / "t.json", tmp_path / "t.csv"
    pj.write_text(JSON, encoding="utf-8")
    pc.write_text(CSV, encoding="utf-8")
    expected = load_tariffs(JSON)
    assert expected[0]["name"] == "a"
    assert load_tariffs(str(pj)) == expected
    assert load_tariffs(pj) == expected
    assert load_tariffs(expected) == expected
    with open(pj, encoding="utf-8") as f:
        assert load_tariffs(f) == expected
    with open(pc, encoding="utf-8") as f:
        assert load_tariffs(f)[0]["name"] == "a"
    csv_rows = load_tariffs(pc)
    assert csv_rows[0] == {
        "name": "a",
        "minutes": "1",
        "data_gb": "1",
        "sms": "1",
        "price": "1",
    }
    assert rank_tariffs(csv_rows) == rank_tariffs(expected)


def test_load_rejects_bad_input() -> None:
    with pytest.raises(TypeError):
        load_tariffs(42)
    with pytest.raises(ValueError):
        load_tariffs([1, 2])
    with pytest.raises(ValueError):
        load_tariffs('{"name": "x"}')

```

11. Тести `total_ordering`: усі шість операторів, `sorted`/`max`, хешованість і `NotImplemented` для рядка; базовий приклад зі студентами:

```python
def test_tariff_ordering() -> None:
    a = Tariff("a", 1, 1.0, 1, 10.0, score=2.0)
    b = Tariff("b", 1, 1.0, 1, 20.0, score=2.0)
    c = Tariff("c", 1, 1.0, 1, 5.0, score=1.0)
    assert c < b < a
    assert a > b and a >= b and c <= b and a != b
    assert a == Tariff("a", 9, 9.0, 9, 10.0, score=2.0)
    assert sorted([a, c, b]) == [c, b, a] and max([a, b, c]) is a
    assert len({a, b, c, Tariff("a", 9, 9.0, 9, 10.0, score=2.0)}) == 3


def test_tariff_not_implemented_for_other_types() -> None:
    t = Tariff("a", 1, 1.0, 1, 10.0)
    assert t.__eq__("x") is NotImplemented
    assert t.__lt__("x") is NotImplemented
    assert t != "x"
    with pytest.raises(TypeError):
        _ = t < "x"


# --- базовий приклад з методички ---


def test_clean_and_rank_students() -> None:
    rows = load_users(
        '[{"name": "  ALICE  ", "age": "19", "score": "8.5"},'
        ' {"name": "bob", "age": "17", "score": "9.1"},'
        ' {"name": "Carol", "age": "21", "score": "7.0"},'
        ' {"name": "denis", "age": "20", "score": "9.3"}]'
    )
    assert clean_and_rank(rows, top=2) == [
        {"name": "Denis", "age": 20, "score": 9.3},
        {"name": "Alice", "age": 19, "score": 8.5},
    ]
```

12. Property-based тести на Hypothesis: Top-N завжди є префіксом повного відсортованого списку, конвеєр не мутує вхід, фільтр за мінімумом хвилин відбирає рівно потрібні записи:

```python

rows_st = st.lists(
    st.fixed_dictionaries(
        {
            "name": st.text(min_size=1, max_size=10),
            "minutes": st.integers(min_value=0, max_value=5000),
            "data_gb": st.floats(min_value=0, max_value=100, allow_nan=False),
            "sms": st.integers(min_value=0, max_value=1000),
            "price": st.floats(min_value=1, max_value=1000, allow_nan=False),
        }
    ),
    max_size=25,
)


@given(rows_st, st.integers(min_value=0, max_value=10))
def test_top_is_prefix_of_sorted(rows: list[dict[str, Any]], n: int) -> None:
    full = rank_tariffs(rows, top=len(rows) + 1)
    assert rank_tariffs(rows, top=n) == full[:n]
    assert full == sorted(full, key=lambda t: (-t.score, t.price, t.name))


@given(rows_st)
def test_no_mutation(rows: list[dict[str, Any]]) -> None:
    original = deepcopy(rows)
    rank_tariffs(rows, top=5, min_minutes=100)
    assert rows == original


@given(rows_st, st.integers(min_value=0, max_value=5000))
def test_filter_respects_min_minutes(rows: list[dict[str, Any]], m: int) -> None:
    top = rank_tariffs(rows, top=100, min_minutes=m)
    assert all(isinstance(t, Tariff) and t.minutes >= m for t in top)
    assert len(top) == sum(1 for r in rows if r["minutes"] >= m)
```

13. Запустив демо з вимогами за замовчуванням і з жорсткішими (500 хв, 20 ГБ, 100 sms): завантажено 4 + 3 + 4 записи з трьох джерел, обрано найкращі за score тарифи:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python main.py
json=4 csv=3 file=4 rows
Business Pro    2000 min  40.0 GB  500 sms  450.00 score=1.389
Student Lite     500 min  15.0 GB  100 sms  180.00 score=1.139
Family Plus     1000 min  25.0 GB  200 sms  320.00 score=1.125
total price of top: 950.00
students: [{'name': 'Alice', 'age': 19, 'score': 8.5}]

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python main.py 500 20 100
json=4 csv=3 file=4 rows
Business Pro    2000 min  40.0 GB  500 sms  450.00 score=1.389
Family Plus     1000 min  25.0 GB  200 sms  320.00 score=1.125
total price of top: 770.00
students: [{'name': 'Alice', 'age': 19, 'score': 8.5}]
```

14. Перевірив проєкт автоматичними інструментами: усі 17 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 17 items

tests\test_lab07.py::test_pipeline_applies_left_to_right PASSED          [  5%]
tests\test_lab07.py::test_mapf_filt_are_partials PASSED                  [ 11%]
tests\test_lab07.py::test_partial_requirements_and_score PASSED          [ 17%]
tests\test_lab07.py::test_operator_helpers PASSED                        [ 23%]
tests\test_lab07.py::test_normalize_creates_new_record PASSED            [ 29%]
tests\test_lab07.py::test_rank_tariffs_top_n_and_order PASSED            [ 35%]
tests\test_lab07.py::test_rank_tariffs_filters_and_tiebreak PASSED       [ 41%]
tests\test_lab07.py::test_total_price_reduce PASSED                      [ 47%]
tests\test_lab07.py::test_no_mutation PASSED                             [ 52%]
tests\test_lab07.py::test_load_from_all_sources PASSED                   [ 58%]
tests\test_lab07.py::test_load_rejects_bad_input PASSED                  [ 64%]
tests\test_lab07.py::test_tariff_ordering PASSED                         [ 70%]
tests\test_lab07.py::test_tariff_not_implemented_for_other_types PASSED  [ 76%]
tests\test_lab07.py::test_clean_and_rank_students PASSED                 [ 82%]
tests\test_properties.py::test_top_is_prefix_of_sorted PASSED            [ 88%]
tests\test_properties.py::test_no_mutation PASSED                        [ 94%]
tests\test_properties.py::test_filter_respects_min_minutes PASSED        [100%]

============================= 17 passed in 1.74s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python -m mypy --strict .
Success: no issues found in 7 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab07> python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я побудував функціональний конвеєр відбору мобільних тарифів (варіант 13) на `functools.partial`, `reduce` та гетерах `operator`: вимоги користувача й ваги оцінки фіксуються частковим застосуванням, сортування виконується за кількома ключами через `attrgetter`, результат — Top-N незмінних об'єктів `Tariff` із повним порядком від `total_ordering`. Поліморфний `load_tariffs` на `singledispatch` дав єдиний інтерфейс для JSON-рядка, шляху, відкритого файлу та готового списку. Юніт- і property-тести, mypy, ruff і black підтвердили коректність конвеєра, завантажувачів і всіх операторів порівняння.